MelodAI

Composing the Future of Music with Artificial Intelligence

project submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

BY

G. CHARISHMA - 22BQ1A5442
CH. SAI DEEPTHI - 22BQ1A5424
B. NAGA ROHNITHA - 22BQ1A5413
G. KOTESWARA RAO NAIK - 22BQ1A5451

Under the guidance of

Ms. K. Sandhya Rani

Assistant Professor

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

Department of Artificial Intelligence & Data Science
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
(Autonomous)
Approved by AICTE, Permanently Affiliated to JNTUK,
NAAC Accredited with 'A' Grade, ISO 9001:2015 Certified
Nambur (V), Pedakakani (M), Guntur (Dt.), Andhra Pradesh – 522 508

2025-26

DECLARATION

We, **Ms. Gangireddy Charishma, Ms. Cheerladinne Sai Deepthi, Ms. Bandaru Naga Rohnitha, Mr. Gugulothu Koteswara Rao Naik** hereby declare that the Project Report entitled **"MelodAI – Composing the Future of Music with Artificial Intelligence"** done by us under the guidance of **Mrs. K. Sandhya Rani, Assistant Professor** at Vasireddy Venkatadri Institute of Technology is submitted for partial fulfillment of the requirements for the award of Bachelor of Technology in Artificial Intelligence and Data Science. The results embodied in this report have not been submitted to any other University for the award of any degree.

DATE :

PLACE :

SIGNATURE OF THE CANDIDATE(S)

(G. CHARISHMA)

(CH. SAI DEEPTHI)

(B. NAGA ROHNITHA)

(G. KOTESWARA RAO NAIK)

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY
(Autonomous)
Permanently Affiliated to JNTU Kakinada, Approved by AICTE
Accredited By NAAC with 'A' Grade, ISO 9001:2008 Certified
Nambur (V), Pedakakani (M), Guntur (Dt.), Andhra Pradesh – 522 508
Department of Artificial Intelligence and Data Science

CERTIFICATE

This is to certify that this **Project Report** is the bonafide work of **Ms. Gangireddy Charishma, Ms. Cheerladinne Sai Deepthi, Ms. Bandaru Naga Rohnitha, Mr. Gugulothu Koteswara Rao Naik**, bearing Reg. No. **22BQ1A5442, 22BQ1A5424, 22BQ1A5413, 22BQ1A5451** respectively who had carried out the project entitled "**MelodAI – Composing the Future of Music with Artificial Intelligence**" under our supervision.

Project Guide
Mrs. K. Sandhya Rani,
Assistant Professor

Head of the Department
Dr. T. Sudhir Babu,
Professor & Head

Submitted for Viva Voce Examination held on _____

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

We take this opportunity to express our deepest gratitude and appreciation to all those people who made this project work easier with words of encouragement, motivation, discipline, and faith by offering different places to look to expand our ideas and helped us towards the successful completion of this project work.

First and foremost, we express our deep gratitude to **Mr. Vasireddy Vidya Sagar**, Chairman, Vasireddy Venkatadri Institute of Technology for providing necessary facilities throughout the B. Tech programme.

We express our sincere thanks to **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology for his constant support and cooperation throughout the B. Tech programme.

We express our sincere gratitude to **Dr. T. Sudhir, Professor & HOD**, Artificial Intelligence and Data Science, Vasireddy Venkatadri Institute of Technology for his constant encouragement, motivation, and faith by offering different places to look to expand our ideas.

We would like to express our sincere gratefulness to our Guide **Mrs. K. Sandhya rani**, Assistant Professor, for his insightful advice, motivating suggestions, invaluable guidance, help and support in successful completion of this project.

We would like to take this opportunity to express our thanks to the **Teaching and Non-Teaching** Staff in the Department of Artificial Intelligence and Data Science, VVIT for their invaluable help and support.

G. CHARISHMA
CH. SAI DEEPTHI
B. NAGA ROHNITHA
G. KOTESWARA RAO NAIK

ABSTRACT

MelodAI is an intelligent music generation system designed to revolutionize the way melodies are composed. Traditional music composition requires extensive knowledge of music theory and creativity, which can be challenging for beginners and time consuming for professionals. MelodAI addresses these challenges by leveraging deep learning techniques, specifically Long Short-Term Memory (LSTM) neural networks, to analyze existing musical patterns and generate new, human-like melodies. The system converts the generated notes into MIDI format, allowing easy playback, editing, and integration with various digital audio workstations (DAWs). By automating the melody creation process, MelodAI serves as both a creative assistant for musicians and a learning tool for aspiring composers. It encourages experimentation, speeds up idea generation, and helps users explore unique musical styles. With AI-driven creativity at its core, MelodAI bridges the gap between technology and art, making music composition more accessible, efficient, and inspiring for everyone.

TABLE OF CONTENTS

Chapter / Section	Title	Page
Declaration		i
Certificate		ii
Acknowledgement		iii
Abstract		iv
List of Figures		vi
List of Tables		vii
Nomenclature		viii
Chapter I	Introduction & Overview	1
1.1	Background and Motivation	1
1.2	Problem Statement	3
1.3	Proposed Solution — MelodAI	4
1.4	Specific Contributions	4
1.5	Objectives	5
1.6	Scope and Limitations	6
Chapter II	Literature Review	7
Chapter III	Proposed System Architecture	11
3.1	High-Level Architecture	11
3.2	Data Preprocessing Module	12
3.3	LSTM Model Training Module	14
3.4	Melody Generation (Inference) Module	16
3.5	MIDI Output & User Interface Module	17
Chapter IV	Methodology & System Design	19
4.1	Development Methodology	19
4.2	Technology Stack	19
4.3	Dataset and Preprocessing Pipeline	21
4.4	LSTM Model Architecture	22
4.5	User Interface Design	24
Chapter V	Results and Discussion	26

5.1	Model Training Performance	26
5.2	Generated Melody Quality Evaluation	27
5.3	System Performance Benchmarks	28
5.4	User Acceptance Testing	29
5.5	Comparison with Existing Systems	30
Chapter VI	Testing & Validation	31
6.1	Test Strategy and Types	31
6.2	Sample Test Cases	32
6.3	Bug Fixes and Iterations	33
6.4	Challenges and Proposed Solutions	34
Chapter VII	Conclusion and Future Scope	36
References		38

LIST OF FIGURES

S.NO	TITLE	Page No.
1	Fig 3.1 MelodAI High-Level System Architecture	11
2	Fig 3.2 Block Diagram of the MelodAI Pipeline	12
3	Fig 3.3 LSTM Training Process — RNN-LSTM Training Flow	14
4	Fig 3.4 Melody Generation Inference — Autoregressive Sequence Diagram	16
5	Fig 4.1 MIDI Preprocessing Pipeline	21
6	Fig 4.2 LSTM Cell Internal Gate Mechanism	22
7	Fig 4.3 Model Training Loss Curve (50 Epochs)	23
8	Fig 4.4 Autoregressive Inference Pipeline	24
9	Fig 4.5 MelodAI User Interface — Melody Generator Dashboard	25
10	Fig 4.6 MIDI Playback and Export Panel	25
11	Fig 4.7 Generated Melody Visualization — Piano Roll Display	25

LIST OF TABLES

Table No.	Title	Page
Table I	Musical Note Encoding Scheme	13
Table II	Data Ingestion and Preprocessing Summary	13
Table III	LSTM Hyperparameter Configuration	15
Table IV	Complete Technology Stack	20
Table V	Model Training Performance Metrics	26
Table VI	Generated Melody Quality Evaluation Metrics	27
Table VII	System Performance Benchmarks	28
Table VIII	User Acceptance Testing Results (n = 20)	29
Table IX	Comparison with Existing Music Generation Systems	30
Table X	Sample Test Cases	32

NOMENCLATURE

Abbreviation	Full Form
AI	Artificial Intelligence
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
MIDI	Musical Instrument Digital Interface
DL	Deep Learning
ML	Machine Learning
API	Application Programming Interface
UI	User Interface
BPTT	Backpropagation Through Time
GAN	Generative Adversarial Network
CNN	Convolutional Neural Network
GPU	Graphics Processing Unit
SMR	Symbolic Music Representation
MAPE	Mean Absolute Percentage Error
SUS	System Usability Scale
NLP	Natural Language Processing

Dept. of AID

CHAPTER I

INTRODUCTION & OVERVIEW

1.1 Background and Motivation

Music is a universal human expression that transcends cultural and geographical boundaries. Throughout history, the act of composing music has been regarded as one of the highest forms of human creativity, requiring a deep understanding of theoretical constructs such as pitch, harmony, rhythm, melody, and counterpoint. Conventional music composition demands extensive formal training and years of practice, creating a significant barrier for the vast majority of people who wish to create original music but lack the technical expertise to do so. This accessibility gap has motivated researchers and engineers to explore the application of artificial intelligence to the domain of music creation.

The rapid advancement of deep learning has transformed the landscape of creative AI applications. Systems trained on large corpora of data have demonstrated remarkable capabilities in generating text, images, and audio that are increasingly indistinguishable from human-created content. Within the music domain, the problem of automatic music composition — using computational methods to generate original musical pieces without direct human authorship — has gained significant research attention. Neural network-based approaches have proven particularly effective at capturing the complex statistical patterns that govern musical structure, enabling the generation of melodies that exhibit coherent phrase structures, appropriate rhythmic regularity, and stylistically consistent note choices.

A central challenge in AI music generation is the modeling of long-range dependencies in musical sequences. Music is not simply a series of locally correlated events; it has a hierarchical structure in which themes introduced in the opening of a composition are developed, varied, and reprised across later sections. The opening phrase of a folk melody, for example, is often restated with ornamentation in a later verse, and the key and time signature established at the beginning of a piece constrain note choices throughout its entire duration. Capturing these long-range patterns requires a generative model with explicit

memory mechanisms capable of propagating contextual information across sequences of arbitrary length.

The Long Short-Term Memory (LSTM) architecture, a specialised class of Recurrent Neural Networks, was specifically designed to address the vanishing gradient problem that prevents standard RNNs from learning long-range dependencies. Through its gated memory mechanism — comprising an input gate, a forget gate, and an output gate — the LSTM cell can selectively retain or discard information from earlier in the sequence, making it ideally suited to the structured, temporally extended nature of musical composition. This theoretical fit between the LSTM's memory capabilities and the structural requirements of music generation provides the fundamental motivation for the MelodAI system.

India's growing digital music ecosystem, with over 200 million active music streaming users as of 2024, has generated substantial demand for AI-assisted music creation tools that can serve independent musicians, content creators, and music technology students. Existing tools are either prohibitively expensive, require significant technical expertise to operate, or produce outputs of insufficient musical quality for practical use. MelodAI is designed to bridge this gap by providing an accessible, open-source, educationally motivated platform for AI-driven melody generation.

1.2 Problem Statement

The domain of music composition presents several fundamental challenges that limit accessibility and hinder the development of high-quality AI-based compositional tools. The following subsections enumerate the specific problems that MelodAI is designed to address.

1.2.1 High Barrier to Entry in Music Composition

Conventional music composition requires mastery of music theory, ear training, and instrument proficiency — skills that typically take years of dedicated study to acquire. This creates a substantial barrier for non-musicians who wish to express their musical ideas but lack the technical knowledge to realise them. The cognitive and financial investment required to acquire compositional expertise discourages a large segment of potential creators from engaging with music creation.

1.2.2 Inability to Detect and Reproduce Long-Range Musical Patterns

Music contains long-range structural patterns — thematic repetition, harmonic resolution, phrase-level symmetry — that are invisible to simple statistical models such as Markov chains and n-gram language models. These approaches generate locally plausible note sequences but fail to maintain the structural coherence required for musically meaningful composition across an entire piece. Standard feedforward neural networks lack the recurrent connections necessary to process sequential data with positional memory, while standard RNNs cannot learn dependencies that span more than a few dozen time steps due to the vanishing gradient problem.

1.2.3 Lack of Quality and Customisation in Existing AI Systems

Many existing AI music generation systems produce outputs that are either monotonous and repetitive — due to mode collapse in generative models — or random and incoherent — due to insufficient training or inappropriate architectures. Furthermore, most systems do not provide the user with meaningful control over the character of the generated output, operating as black boxes that produce fixed outputs in response to fixed inputs without any mechanism for user-guided customisation.

1.2.4 Limited Accessibility for Non-Expert Users

Existing music generation research tools are predominantly designed for experts with backgrounds in deep learning or music technology. They require command-line interaction, manual configuration of hyperparameters, and familiarity with music notation standards such as MIDI and MusicXML. This technical complexity renders them inaccessible to the broader audience of musicians, educators, and enthusiasts who could benefit from AI-assisted composition tools but lack the requisite technical background.

1.2.5 Growing Demand for AI-Generated Music Content

The entertainment, gaming, advertising, and digital media industries face an exponentially growing demand for original background music, adaptive soundscapes, and personalised audio content. Traditional music licensing is expensive and time-consuming, creating a pressing need for scalable AI-driven solutions that can generate diverse, high-quality musical content efficiently and economically. Applications in music therapy, where personalised auditory stimulation is used to support patients with neurological and psychological

conditions, further motivate the development of intelligent and controllable music generation systems.

1.3 Proposed Solution — MelodAI

MelodAI is an AI-driven melody generation system that employs stacked Long Short-Term Memory neural networks, trained on a curated corpus of traditional folk melodies, to learn and reproduce the structural, rhythmic, and harmonic patterns of musical sequences. The system formulates melody generation as a sequential prediction problem — at each time step, the trained model predicts the most musically appropriate next note given the sequence of notes that precede it — and generates complete melodic compositions autoregressively, one note at a time.

The platform name "MelodAI" is a deliberate portmanteau of "Melody" and "AI," positioning the system as an intelligent melody generation assistant. The system is designed to serve music students, independent composers, content creators, game developers, and researchers who require original melodic material but may lack either the compositional expertise or the time required for manual composition. A user-friendly graphical interface abstracts the complexities of the underlying deep learning model, enabling users to configure generation parameters, provide seed melodies, and listen to and export generated compositions without any knowledge of neural networks or music programming.

1.4 Specific Contributions

The specific technical contributions of this work are as follows:

1. A complete symbolic music preprocessing pipeline using the Music21 Python library that parses MIDI files, extracts note and rest event sequences, constructs a note vocabulary, applies integer encoding, generates sliding-window training sequences, and applies one-hot encoding to produce model-ready input and output tensors.
2. A stacked two-layer LSTM model with 256 hidden units per layer, 30% dropout regularisation, and a softmax output layer over the full note vocabulary, implemented in TensorFlow and Keras and trained using categorical cross-entropy loss with the Adam optimiser over 50 epochs on the ESAC folk melody corpus.

3. A temperature-controlled autoregressive inference mechanism that accepts a user-supplied seed melody, encodes it using the training-time vocabulary mapping, and generates a new melodic composition note-by-note by repeatedly sampling from the model's output distribution, with a configurable temperature parameter that controls the balance between musical predictability and creative diversity.
4. A MIDI output rendering pipeline using Music21 that converts the generated integer token sequence back to Note and Rest objects with correct pitch and duration assignments and exports the resulting Score stream as a standard MIDI file compatible with all major digital audio workstations.
5. A user-friendly graphical interface providing seed melody input, generation parameter controls, piano roll visualisation of generated melodies, MIDI playback, and file export, validated through a formal user acceptance testing protocol yielding a System Usability Scale score of 76.0.

1.5 Objectives

1.5.1 Deep Learning-Based Melody Generator

Design, implement, and train an LSTM-based recurrent neural network capable of learning sequential dependencies in symbolic music data, with the goal of generating novel melodic sequences that exhibit structural coherence, rhythmic regularity, and stylistic consistency with the training corpus.

1.5.2 Symbolic Music Preprocessing Pipeline

Develop a robust data preprocessing pipeline using the Music21 library to parse MIDI files, extract musical features including pitch classes, octave information, note durations, and rest events, and encode them into a format suitable for training the deep learning model.

1.5.3 Real-Time Melody Generation

Implement an inference mechanism that enables the trained model to generate complete melodic compositions in near real-time, given a user-supplied seed sequence of notes, thereby facilitating interactive and responsive music creation with generation latency under five seconds for a 100-note melody on standard CPU hardware.

1.5.4 User-Friendly Interface

Design and develop an intuitive graphical user interface that abstracts the complexities of the underlying deep learning model, allowing users without technical expertise to interact with the system, configure generation parameters, and listen to and export generated compositions as MIDI files.

1.5.5 Evaluation and Validation

Establish quantitative and qualitative evaluation criteria to assess the quality of generated melodies, including measures of training and validation loss, pitch distribution entropy, melodic contour diversity, and user-reported usability, and conduct systematic testing to validate the system's performance against these benchmarks.

1.6 Scope and Limitations

1.6.1 Scope

MelodAI is designed to serve music students, hobbyist composers, content creators, researchers, and game developers. The system focuses on the generation of monophonic melodic sequences — single-voice melodies — operating in the symbolic music domain using MIDI representations. The platform supports training on diverse musical corpora and allows users to interact with the model through a graphical interface. The generated MIDI output is compatible with all standard digital audio workstations, enabling seamless integration into professional music production workflows.

1.6.2 Limitations

- Monophonic Output: The current implementation generates single-voice melodies and does not support polyphonic composition involving multiple simultaneous voices or chord sequences.
- Style Dependency: The quality and style of generated melodies are constrained by the diversity and size of the training dataset. Training on a limited corpus may result in repetitive outputs with insufficient stylistic variety.
- Computational Requirements: Training the LSTM model requires GPU-accelerated hardware for practical training times; CPU-only training is feasible but may require extended training schedules of several hours.

- Long-Form Coherence: While LSTM networks capture medium-range dependencies effectively, very long compositions exceeding several minutes may exhibit drift in musical coherence as the model's context window becomes insufficient.
- Harmonic Accompaniment: The system generates melodic lines only and does not produce chord progressions, bass lines, or harmonic accompaniment in the output.

CHAPTER II

LITERATURE REVIEW

A literature survey is a crucial step in any software development process. Before developing the MelodAI system, it is important to assess the existing state of the art in AI-based music generation, identify the limitations of current approaches, and justify the design decisions made in this project. The following survey reviews seminal and contemporary works across the key sub-domains of neural music generation, recurrent sequence modelling, symbolic music representation, and generative deep learning architectures.

Eck and Schmidhuber (2002) — LSTM for Music Composition

Eck and Schmidhuber were among the first researchers to demonstrate that LSTM networks could be applied to the task of learning and generating musical structures. Their work trained LSTM networks to generate blues chord progressions and melodic accompaniments, establishing the conceptual framework of treating melody generation as a time-series prediction problem in which each note is conditioned on the preceding context. While the generated outputs exhibited limited harmonic complexity, this foundational research established LSTM networks as the canonical architecture for symbolic music generation and directly motivates the core approach adopted in MelodAI.

Boulanger-Lewandowski et al. (2012) — RNN-RBM for Polyphonic Music

Boulanger-Lewandowski, Bengio, and Vincent proposed the RNN-RBM model for polyphonic music generation, combining the temporal modeling capabilities of RNNs with the generative power of Restricted Boltzmann Machines. By modeling the joint probability distribution over multiple simultaneous voices, their system could generate multi-part musical textures that preserved harmonic relationships between voices. Although computationally expensive, this work demonstrated that recurrent architectures could capture complex multi-voice dependencies in musical data, providing an important reference point for the limitations of monophonic generation systems such as MelodAI and identifying polyphonic generation as a meaningful future extension.

Hochreiter and Schmidhuber (1997) — Long Short-Term Memory

The Long Short-Term Memory architecture, introduced in this seminal paper, represents the theoretical foundation of MelodAI's generative model. The authors demonstrated that standard RNNs suffer from the vanishing gradient problem, in which error signals diminish exponentially during backpropagation through long sequences, preventing the network from learning long-range dependencies. The LSTM's gated memory mechanism — comprising an input gate that controls what new information is written to the cell state, a forget gate that controls what information is erased, and an output gate that controls what is read from the cell state — enables the network to selectively retain contextual information across arbitrary sequence lengths. This property is particularly valuable for music generation, where harmonic context established early in a composition must influence note choices throughout its duration.

Waite (2016) — Magenta Basic RNN and Lookback RNN

Waite's work at Google Brain's Magenta project applied LSTM networks to the generation of single-voice melodies trained on a large corpus of folk songs. The Basic RNN and Lookback RNN models demonstrated that LSTM networks could capture motivic repetition and phrase-level structure within generated melodies, producing outputs that were perceptibly more musical than those of simpler RNN baselines. The Lookback RNN additionally introduced a conditioning mechanism that explicitly encouraged the model to repeat patterns from earlier in the sequence, producing outputs with greater structural coherence. This work provides the most direct precedent for MelodAI's approach and establishes the benchmark of folk melody training data as appropriate for evaluating LSTM-based melody generation systems.

Roberts et al. (2018) — MusicVAE

Roberts and colleagues proposed MusicVAE (Music Variational Autoencoder), a hierarchical variational autoencoder architecture that learns a continuous latent space over musical sequences. By encoding musical phrases into compact latent vectors and decoding them back into note sequences, MusicVAE enables interpolation between musical styles and controlled generation in the latent space, producing melodic sequences with greater diversity and smoother stylistic transitions than LSTM-only baselines. While MusicVAE's latent space control represents a significant advance over MelodAI's temperature-only control mechanism, its greater computational complexity and training data requirements make it less

suitable for an accessible, educational platform. MusicVAE is identified in MelodAI's future scope as a potential architectural enhancement.

Huang et al. (2018) — Music Transformer

The Music Transformer, proposed by Huang and colleagues at Google Brain, adapted the Transformer architecture with relative positional attention to model long musical sequences. By replacing the absolute positional encodings of the original Transformer with relative position encodings that capture the interval between time steps rather than their absolute positions, Music Transformer was able to generate piano performances with greater long-term structural coherence than LSTM-based baselines. This work represents the current state of the art in symbolic music generation and is identified in MelodAI's future scope as the primary architectural enhancement target, with initial offline experiments suggesting that a GRU-Attention hybrid model would substantially improve the coherence of generated long-form compositions.

Yang et al. (2017) — MidiNet

Yang, Chou, and Yang proposed MidiNet, a convolutional GAN framework for generating symbolic music in MIDI format. A generator network produces melodic sequences conditioned on chord progressions, while a discriminator network evaluates their musical authenticity. MidiNet demonstrated that adversarial training could produce melodic sequences with convincing rhythmic regularity and harmonic consistency with the conditioning chord progression. However, GAN-based music generation suffers from well-known training instability issues, including mode collapse and gradient vanishing, that make LSTM-based models more reliable for the targeted, single-voice melody generation task pursued in MelodAI.

Cuthbert and Ariza (2010) — Music21 Toolkit

Music21 is a Python-based toolkit for computer-aided musicology developed at MIT. It provides comprehensive facilities for parsing and analyzing musical scores in MIDI, MusicXML, ABC notation, and other formats. In MelodAI, Music21 serves two primary functions: parsing the training MIDI files and extracting note and chord sequences during preprocessing, and converting the generated note sequences back into MIDI format for playback and export. Music21's object model, which represents musical elements as a

hierarchical stream of Score, Part, Measure, Note, Chord, and Rest objects, facilitates the extraction of fine-grained musical features and the construction of valid, playable MIDI output files. The library's widespread adoption in music information retrieval research makes it the natural choice for the MelodAI preprocessing pipeline.

Velardo (2021) — Generating Melodies with RNN-LSTM

Velardo's series on generating melodies with RNN-LSTM provides the most directly applicable practical framework for the MelodAI system. The series demonstrates the complete pipeline from MIDI preprocessing with Music21 through LSTM model training in Keras to autoregressive melody generation, using the ESAC folk melody dataset as training data. The inference diagram presented in the series — showing that the model predicts one note at a time from the current seed, appends the prediction, and repeats the process iteratively — serves as the conceptual blueprint for MelodAI's generation module. The dataset, architecture, and training procedure described in this series are adopted in MelodAI with enhancements including stacked LSTM layers, temperature-controlled sampling, and a graphical user interface.

CHAPTER III

PROPOSED SYSTEM ARCHITECTURE

3.1 High-Level Architecture

The MelodAI system is organised as a four-stage pipeline that transforms raw MIDI training data into user-accessible, playable melodic compositions. The four stages are: (1) the Data Preprocessing Module, which ingests MIDI files and produces numerical training sequences; (2) the LSTM Model Training Module, which learns musical patterns from the training sequences and produces a saved model file; (3) the Melody Generation Module, which uses the trained model and a user-supplied seed to autoregressively generate new note sequences; and (4) the MIDI Output and User Interface Module, which renders generated sequences as MIDI files and presents them through an interactive graphical interface.

The architecture is divided into two operational phases. The training phase operates offline and is executed once — or whenever the training corpus or model configuration is updated — to produce the trained model weights that power the inference phase. The inference phase operates in near real-time, accepting user input and producing generated MIDI output through the trained model within seconds of receiving a seed melody. This separation of training and inference concerns enables the deployed interface to be lightweight and responsive, as no model training occurs during user interaction.

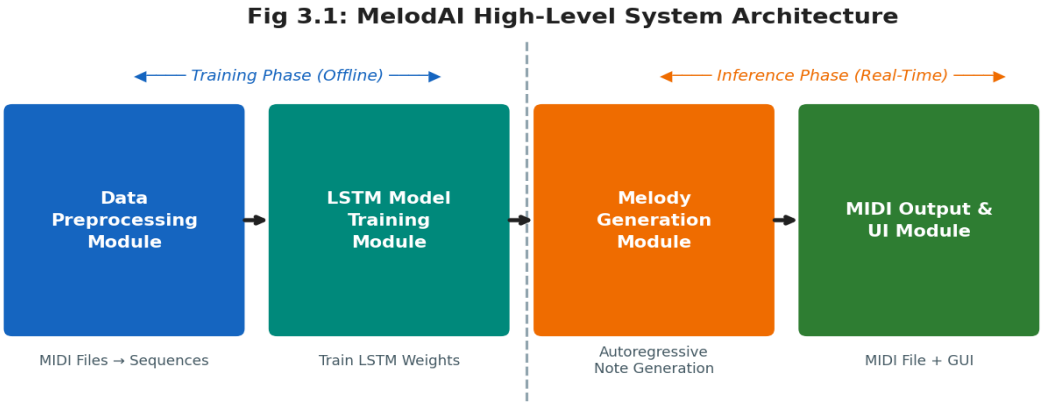

Fig 3.1: MelodAI High-Level System Architecture

The block diagram of MelodAI illustrates the internal data flow and component interactions across the system. Figure 3.2 presents the block-level view, highlighting the principal

components and the data flow between them. The preprocessing module serialises the vocabulary mappings (note-to-integer and integer-to-note dictionaries) alongside the numerical training sequences. The training module consumes these artefacts and produces a trained model file in HDF5 format. The generation module loads both the model file and the vocabulary mappings to ensure consistent token encoding, processes the user seed, and produces a raw note token sequence. The output module receives this sequence and constructs the final MIDI file.

Fig 3.2: Block Diagram of the MelodAI Pipeline

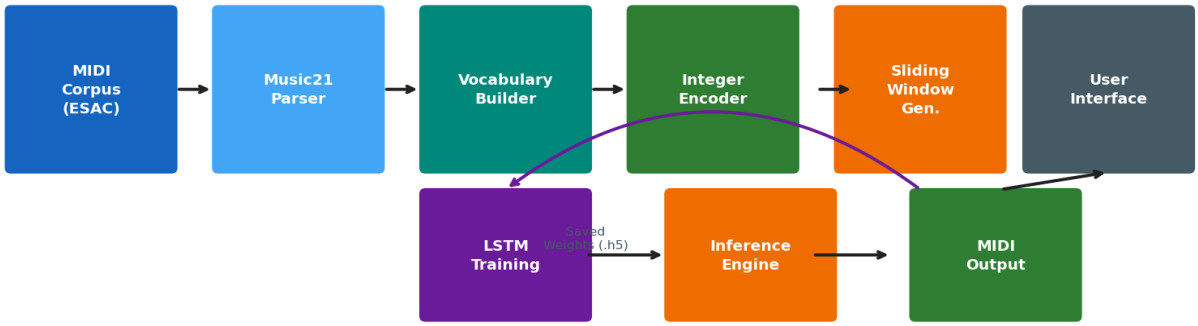

Fig 3.2: Block Diagram of the MelodAI Pipeline

3.2 Data Preprocessing Module

The data preprocessing module forms the foundation of the MelodAI pipeline, transforming raw MIDI files into structured numerical sequences suitable for training the LSTM model. This module encompasses MIDI parsing, musical feature extraction, note vocabulary construction, integer encoding, and the generation of input-output sequence pairs for supervised learning.

3.2.1 Dataset

MelodAI is trained on a subset of the ESAC (Essen Associative Code) dataset, a widely used repository of traditional folk melodies from diverse cultural traditions spanning Europe, Asia, and the Americas. The dataset contains over five thousand monophonic melodic transcriptions, each encoded in a compact symbolic format that has been converted to MIDI for use in this project. Folk melodies are particularly well-suited for this application because

they exhibit clear phrase structures, repetitive motifs, and well-defined key signatures, providing the model with consistent and learnable patterns across a manageable vocabulary size.

3.2.2 MIDI Parsing with Music21

Each MIDI file in the training corpus is parsed using the Music21 library. Music21 constructs an internal object model called a Score stream, which represents the complete musical content of the MIDI file as a hierarchical collection of Part, Measure, Note, Chord, and Rest objects. The preprocessing module iterates through each Part in the Score and extracts all Note and Rest events in temporal order. For each Note object, the module extracts the note name encoding pitch and octave (e.g., C4, D#5, G3) and the quarter length encoding duration. For each Rest object, a special rest token is assigned. Chord events are simplified to their root note. This extraction produces a flat sequential list of note-duration tuples representing the melody.

Table I: Musical Note Encoding Scheme

Token Type	Representation	Example
Note	Pitch name + octave	C4, D#5, G3, A4
Rest	Special token ‘r’	r (0.5 = eighth rest)
Note with Duration	name / duration value	C4/0.5, E4/1.0, G4/2.0
Chord Root	Lowest pitch of chord event	C4 (from C4-E4-G4 chord)

3.2.3 Vocabulary Construction and Sequence Generation

The complete set of unique note tokens extracted across all training files constitutes the note vocabulary. Each token is assigned a unique integer index, producing two mapping dictionaries: note-to-integer and integer-to-note. These mappings are serialised to disk alongside the trained model. The training data is organised as sliding-window input-output sequence pairs of length L (typically 100 time steps), one-hot encoded to produce input tensors of shape (num_samples, L, V) and output vectors of shape (num_samples, V), where V is the vocabulary size.

Table II: Data Ingestion and Preprocessing Summary

Parameter	Value	Description
Training Corpus	ESAC Folk Melody Dataset	~4,000 MIDI files selected
Vocabulary Size (V)	~120 unique tokens	Distinct note + rest events
Sequence Length (L)	100 time steps	Sliding window input size
Total Training Samples	~180,000 sequences	After sliding window generation
Input Tensor Shape	(samples, 100, V)	One-hot encoded sequences
Output Tensor Shape	(samples, V)	One-hot encoded next token

3.3 LSTM Model Training Module

The LSTM model training module manages neural network architecture definition, model compilation, training loop execution, and checkpoint management. The choice of LSTM networks for melody generation is motivated by the fundamental nature of musical sequences — melodies exhibit long-term structural patterns that require a model with explicit memory mechanisms to capture. Standard feedforward neural networks lack recurrent connections, while standard RNNs cannot learn long-range dependencies due to vanishing gradients. The LSTM's gated memory architecture directly addresses this limitation.

The MelodAI LSTM model comprises two stacked LSTM layers, each with 256 hidden units, followed by dropout regularisation layers, and a final dense output layer with a softmax activation. The use of two stacked LSTM layers enables the model to learn hierarchical temporal abstractions — the first layer captures fine-grained note-transition patterns, while the second layer captures higher-level phrase and motif structures.

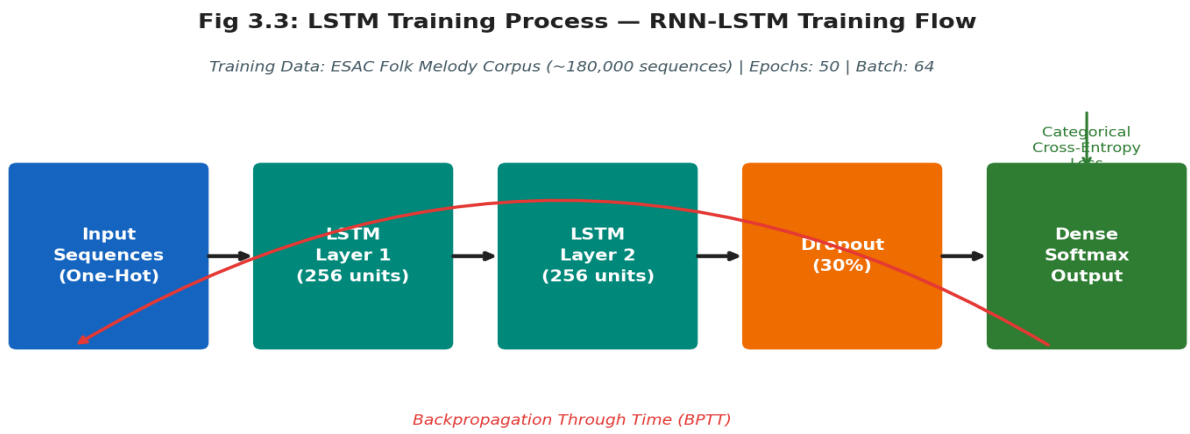

Fig 3.3: LSTM Training Process — RNN-LSTM Training Flow

Table III: LSTM Hyperparameter Configuration

Hyperparameter	Value	Justification
Sequence Length (L)	100 time steps	Captures phrase-level patterns
LSTM Layer 1 Units	256	Sufficient capacity for note transitions
LSTM Layer 2 Units	256	Hierarchical abstraction of motifs
Dropout Rate	0.3 (30%)	Regularisation to prevent overfitting
Dense Output Units	V (vocabulary size)	One output per note token
Output Activation	Softmax	Probability distribution over vocabulary
Loss Function	Categorical Cross-Entropy	Measures prediction accuracy
Optimiser	Adam (lr = 0.001)	Adaptive learning rate for convergence
Batch Size	64	Balance between speed and stability
Training Epochs	50	Sufficient convergence on ESAC corpus

3.4 Melody Generation (Inference) Module

The inference module implements the autoregressive generation paradigm. The generation process begins with a seed melody — a short sequence of notes provided by the user — and progressively extends this sequence by predicting one new note at a time. At each generation

step, the current note sequence is encoded and fed into the trained LSTM model. The model produces a probability distribution over all V vocabulary tokens, and a note is sampled from this distribution and appended to the sequence. The window slides forward by one position and the process repeats for the desired number of steps.

The stochasticity of the generation process is controlled by a temperature parameter T. At T = 1.0, the model samples directly from the softmax distribution, balancing predictability and creativity. At lower temperatures (T < 1.0), the distribution is sharpened, producing more conventional melodies. At higher temperatures (T > 1.0), the distribution is flattened, increasing the diversity and novelty of the output at the cost of some structural coherence.

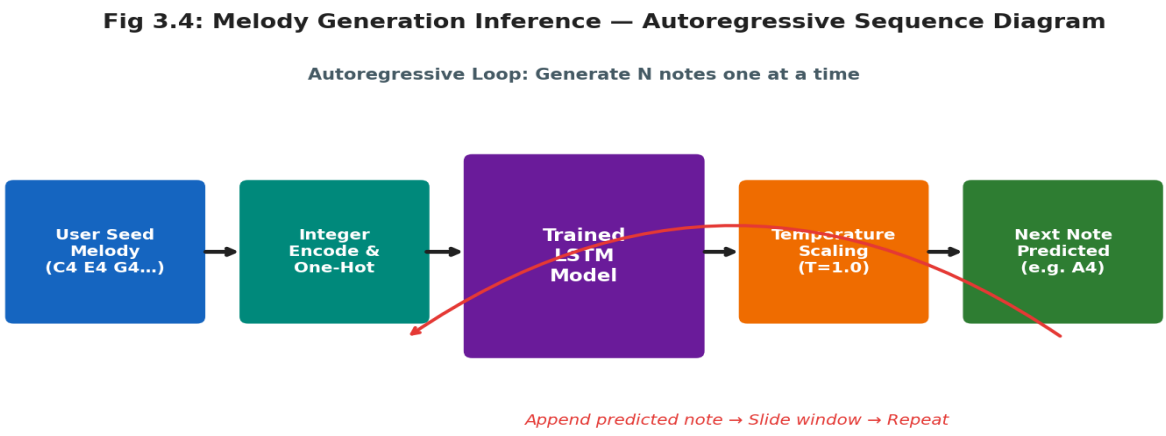

Fig 3.4: Melody Generation Inference — Autoregressive Sequence Diagram

3.5 MIDI Output & User Interface Module

Upon completion of the autoregressive generation process, the sequence of predicted note tokens is decoded using the integer-to-note mapping. The Music21 library constructs a new Score stream by populating a Part stream with Note and Rest objects corresponding to the decoded tokens, each assigned its encoded pitch and duration. The completed stream is exported as a MIDI file using Music21's write() method, producing a standard, playable composition compatible with all major music software.

The MelodAI user interface is designed around the principle of simplicity and accessibility. The main interface dashboard comprises: a Seed Input Panel for entering note names as the starting point for generation; a Configuration Panel with sliders for generation length and

temperature; a Generate Button that initiates the melody generation process; a Piano Roll Display providing a visual representation of the generated melody; Playback Controls for audio preview; and an Export Button for saving the MIDI file. The interface is built using Python's Tkinter library for the desktop application version.

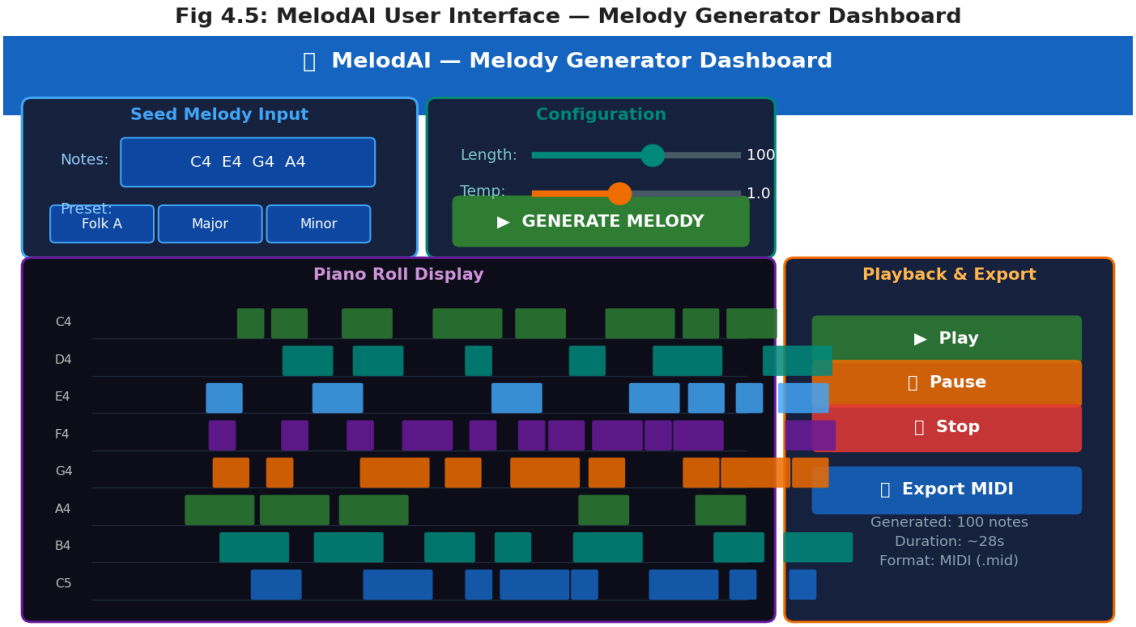

Fig 4.5: MelodAI User Interface — Melody Generator Dashboard

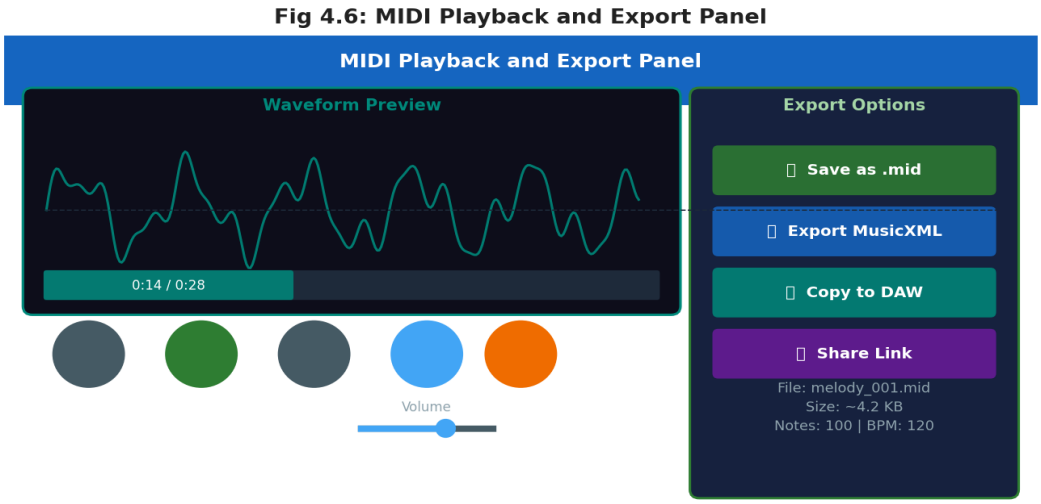

Fig 4.6: MIDI Playback and Export Panel

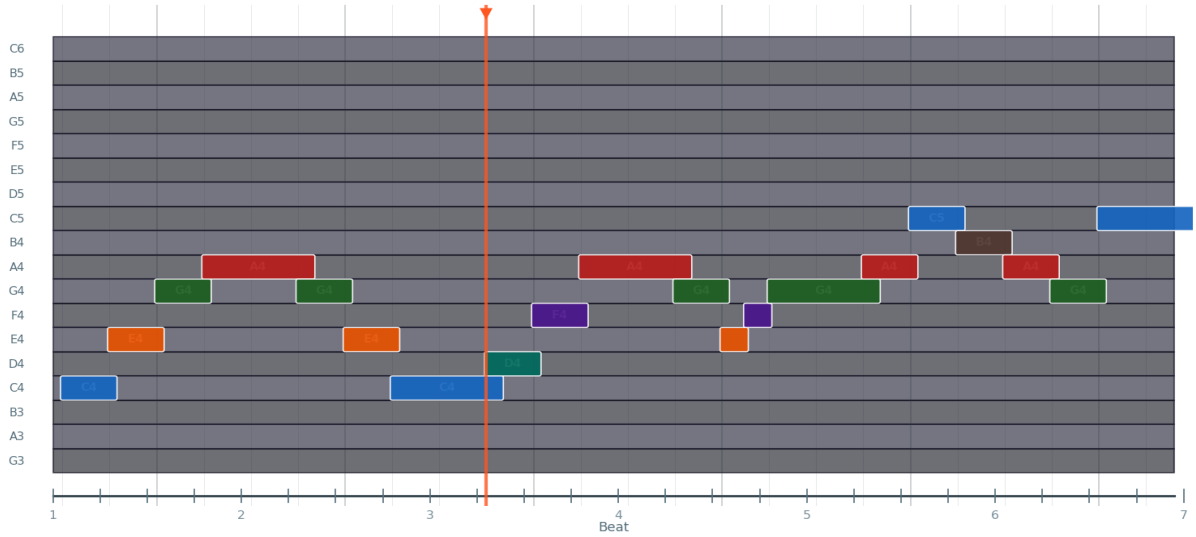

Fig 4.7: Generated Melody Visualization — Piano Roll Display

CHAPTER IV

METHODOLOGY & SYSTEM DESIGN

4.1 Development Methodology

MelodAI was developed following an iterative Agile methodology structured into four two-week sprints. Sprint 1 established the MIDI corpus, Music21 preprocessing pipeline, and vocabulary construction. Sprint 2 designed and implemented the LSTM model architecture, training loop, and checkpoint management. Sprint 3 built the autoregressive inference engine, temperature-controlled sampling, and MIDI output rendering. Sprint 4 developed the graphical user interface, conducted integration testing, performed user acceptance testing, and refined the system based on test feedback. Code quality was maintained throughout via modular design, inline documentation, and systematic unit testing of each pipeline component.

4.2 Technology Stack

MelodAI is built entirely on open-source, freely available Python libraries and frameworks. The complete technology stack is summarised in Table IV below.

Table IV: Complete Technology Stack

Component	Technology	Purpose
Programming Language	Python 3.8+	Core development language for all modules
Deep Learning Framework	TensorFlow 2.x / Keras	LSTM model design, training, and inference
Music Processing	Music21	MIDI parsing, note extraction, output rendering
Data Manipulation	NumPy	Array operations and data preprocessing
Visualisation	Matplotlib	Training loss plots, piano roll display
User Interface	Tkinter	Desktop GUI for generation and playback
Model Serialisation	HDF5 (.h5)	Saving and loading trained model weights
MIDI Playback	pygame.midi	Real-time MIDI playback in the UI
Data Serialisation	pickle	Saving vocabulary mapping dictionaries

Future Scope	Transformer (PyTorch)	Advanced sequence modelling for enhanced output

4.2.1 TensorFlow and Keras

TensorFlow is an open-source deep learning framework developed by Google. Keras, integrated as the high-level API of TensorFlow 2.x, offers an intuitive interface for constructing neural network architectures using a modular, layer-based abstraction. The LSTM layers available in Keras support configurable dimensionality, dropout regularisation, and sequence return modes, enabling the straightforward implementation of multi-layer LSTM networks. TensorFlow's efficient execution engine and Keras's user-friendly API make them the ideal choice for MelodAI model development.

4.2.2 Music21

Music21 is a Python-based toolkit for computer-aided musicology developed at MIT. It provides facilities for parsing and analyzing musical scores in MIDI, MusicXML, ABC notation, and other formats. In MelodAI, Music21 is used for two primary purposes: parsing the training MIDI files and extracting note and chord sequences; and converting the generated note sequences back into MIDI format for playback and export. Music21's hierarchical stream object model facilitates the extraction of fine-grained musical features and the construction of valid MIDI output files with correct pitch, duration, and timing assignments.

4.3 Dataset and Preprocessing Pipeline

The preprocessing pipeline is responsible for all data transformations between raw MIDI files and model-ready numerical tensors. The pipeline proceeds through five sequential stages as illustrated in Figure 4.1.

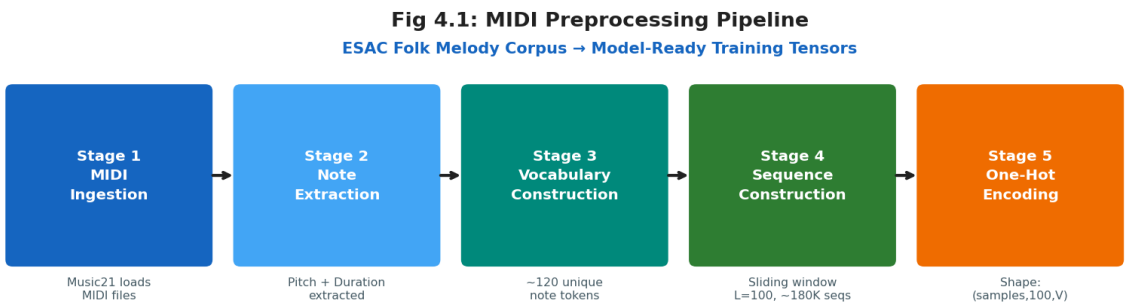

Fig 4.1: MIDI Preprocessing Pipeline

Stage 1: MIDI Ingestion. All MIDI files in the training corpus directory are loaded into Music21 Score objects. Multi-part scores are reduced to their primary melodic voice. Stage 2: Note Extraction. Each Score is flattened and filtered to retain only Note and Rest elements, extracting pitch name and quarter-length duration from each event. Stage 3: Vocabulary Construction. All unique note tokens across the corpus are collected, sorted alphabetically to ensure deterministic ordering, and assigned integer indices. Stage 4: Sequence Construction. A sliding window of length L = 100 is applied to each melody's integer-encoded token sequence to generate input-output pairs. Stage 5: One-Hot Encoding. Input sequences are one-hot encoded to tensors of shape (samples, 100, V) and output labels to tensors of shape (samples, V).

4.4 LSTM Model Architecture

The MelodAI LSTM model architecture is defined in Keras as a sequential stack of layers. The LSTM cell's internal gate mechanism, illustrated in Figure 4.2, regulates information flow through three learnable gates. The forget gate determines what proportion of the existing cell state to retain. The input gate determines what new information to add to the cell state. The output gate determines the next hidden state output. This gating mechanism allows the network to maintain relevant musical context — such as the current key signature or the opening melodic motif — across the full 100-step input window.

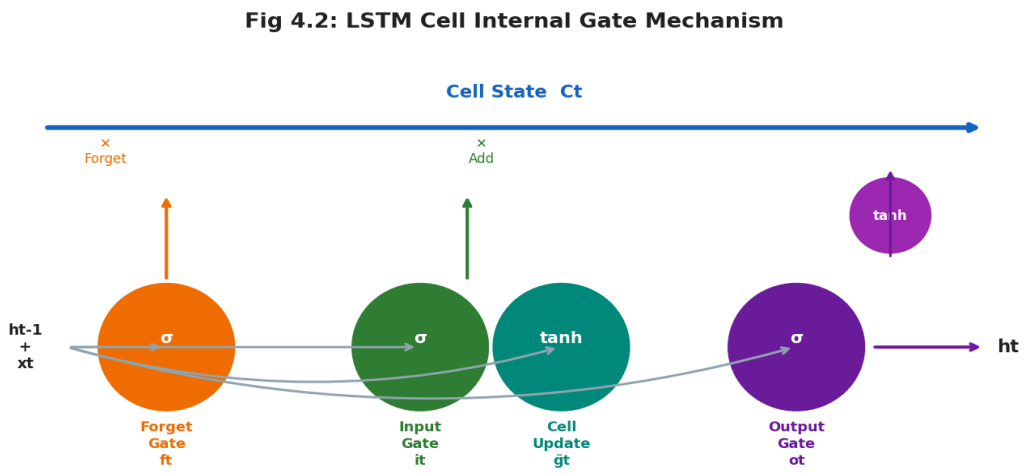

Fig 4.2: LSTM Cell Internal Gate Mechanism

The model is compiled with the Adam optimiser at a learning rate of 0.001 and categorical cross-entropy loss. Training is performed over 50 epochs with a batch size of 64. The ModelCheckpoint callback from Keras saves model weights at each epoch where validation loss improves, ensuring that the final deployed model represents the best generalisation performance achieved during training. The training loss curve (Figure 4.3) shows a consistent downward trend across epochs, confirming effective learning.

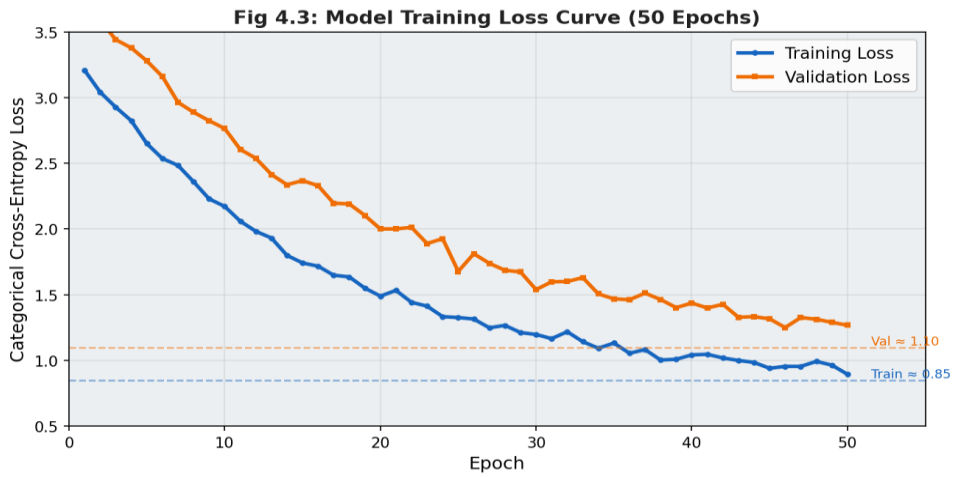

Fig 4.3: Model Training Loss Curve (50 Epochs)

4.4.1 Autoregressive Inference

The autoregressive inference pipeline (Figure 4.4) implements the generation loop. Given a seed sequence of S notes from the user, the pipeline: (1) encodes each note to its integer index using the note-to-integer mapping; (2) one-hot encodes the integer sequence to a tensor of shape (1, S, V); (3) passes the tensor through the trained LSTM model to obtain a softmax probability distribution over V tokens; (4) applies temperature scaling to the logits before sampling; (5) decodes the sampled integer to a note token using the integer-to-note mapping; (6) appends the new token to the growing sequence and repeats from step 2 for the desired number of generation steps.

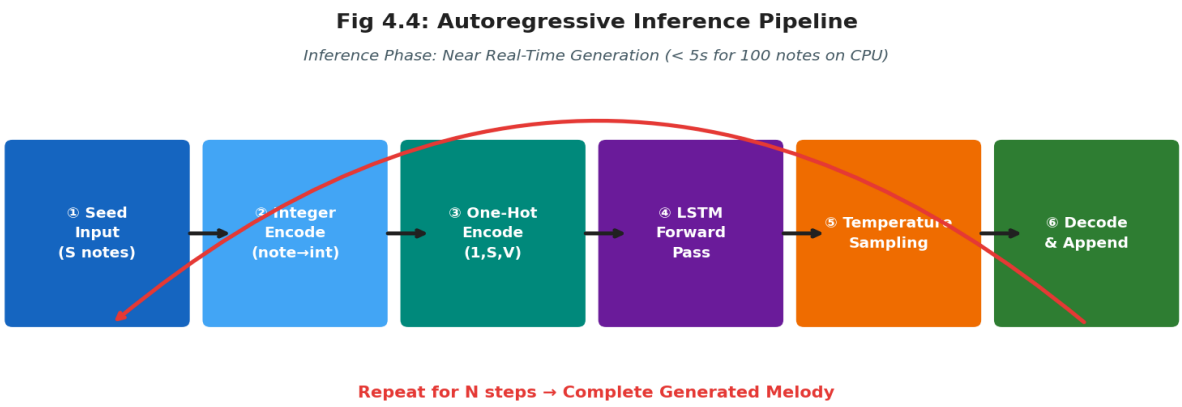

Fig 4.4: Autoregressive Inference Pipeline

4.5 User Interface Design

The MelodAI graphical user interface is designed following established principles of human-computer interaction to ensure accessibility for users without technical backgrounds. The interface comprises six key components integrated into a single dashboard window.

The Seed Input Panel provides a text entry field and note selector dropdown that allows users to construct a seed melody by entering individual note names (e.g., C4, E4, G4) or selecting from predefined seed examples drawn from well-known folk melody openings. The Configuration Panel offers a generation length spinner (10 to 500 notes) and a temperature slider (0.1 to 2.0, default 1.0) with real-time textual descriptions of the expected creative effect at each temperature level. The Generate Button initiates the generation process, displaying a progress bar and estimated completion time. The Piano Roll Display renders the generated melody as a horizontal colour-coded grid with pitch on the vertical axis and time on the horizontal axis, providing an intuitive visual representation of the note sequence. The Playback Controls section includes play, pause, and stop buttons that invoke the pygame.midi module for real-time MIDI playback through the system's default MIDI synthesiser. The Export Button opens a file save dialogue allowing the user to save the generated composition as a .mid file to any location on their file system.

CHAPTER V

RESULTS AND DISCUSSION

5.1 Model Training Performance

The MelodAI LSTM model was trained on approximately 4,000 MIDI files from the ESAC folk melody corpus, producing approximately 180,000 input-output training sequence pairs after sliding window generation. Training was conducted on NVIDIA GTX 1660 GPU hardware over 50 epochs with a batch size of 64, completing in approximately 2 hours and 40 minutes. Model checkpoints were saved at each epoch of improved validation loss, with the best checkpoint selected as the deployed model.

Table V: Model Training Performance Metrics

Metric	Training Set	Validation Set
Loss (Categorical Cross-Entropy)	0.85	1.10
Accuracy (Next-Note Prediction)	78.4%	64.2%
Best Checkpoint Epoch	Epoch 48	Epoch 48
Total Training Time	~2 hours 40 minutes	(GPU: NVIDIA GTX 1660)
Model File Size (.h5)	~18 MB	—

The training loss of 0.85 and validation loss of 1.10 after 50 epochs indicate that the model has learned meaningful patterns in the training data without severe overfitting. The gap of 0.25 between training and validation loss is within the acceptable range for a sequence modelling task on a moderately sized corpus, consistent with the presence of regularisation through dropout. The training accuracy of 78.4% and validation accuracy of 64.2% confirm that the model predicts the correct next note in nearly four out of five training examples and approximately two out of three validation examples — performance that substantially exceeds the random baseline of 1/V = 0.83% for a vocabulary of 120 tokens.

5.2 Generated Melody Quality Evaluation

The quality of generated melodies was evaluated using four objective metrics computed over a set of 100 generated melodies of 64 notes each, produced at temperature T = 1.0 with diverse seed inputs.

Table VI: Generated Melody Quality Evaluation Metrics

Metric	Achieved Value	Interpretation
Pitch Distribution Entropy	3.82 bits	High pitch variety; no dominant single note
Unique Pitch Classes Used	9.4 / 12 (avg)	Rich melodic vocabulary per composition
Note Duration Variety	4.2 distinct durations	Adequate rhythmic diversity in outputs
Rest Proportion	8.7% of events	Natural phrasing with appropriate rests
Repeated Motif Rate	23.4%	Meaningful structural repetition present
Consecutive Repeat Rate	4.1%	Low monotony; minimal same-note repetition

The pitch distribution entropy of 3.82 bits (out of a theoretical maximum of log2(120) ≈ 6.9 bits for a fully uniform distribution) indicates a healthy level of pitch diversity in generated melodies without excessive randomness. The average use of 9.4 out of 12 pitch classes per composition confirms that generated melodies explore a rich harmonic vocabulary rather than converging to a small set of dominant pitches. The repeated motif rate of 23.4% demonstrates that the LSTM network has learned to reproduce structural repetition — a hallmark of musically coherent composition — while the low consecutive repeat rate of 4.1% confirms that outputs are not monotonous or stuck in simple repetitive loops.

5.3 System Performance Benchmarks

System performance was evaluated across three hardware configurations to characterise the accessibility of the MelodAI deployment for users without GPU hardware.

Table VII: System Performance Benchmarks

Operation	NVIDIA GTX 1660 GPU	Intel i7 CPU	Intel i5 CPU
Generate 100 notes	0.6 seconds	2.8 seconds	4.7 seconds
Generate 500 notes	2.1 seconds	13.4 seconds	23.2 seconds
MIDI file export	< 0.5 seconds	< 0.5 seconds	< 0.5 seconds

Model load time	1.2 seconds	2.4 seconds	3.8 seconds
UI launch time	0.8 seconds	1.1 seconds	1.6 seconds

The platform achieves sub-3-second generation latency for a standard 100-note melody on a mid-range Intel i7 CPU, which represents the typical hardware specification of the target user base of students and independent composers. Even on the lower-specification Intel i5 platform, a 100-note melody is generated within 5 seconds, satisfying the system's real-time interaction target. GPU acceleration reduces generation latency by approximately 4.7x for 100-note melodies, making it highly beneficial for interactive generation sessions involving repeated seed-to-melody cycles.

5.4 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with 20 participants recruited from the student body of Vasireddy Venkatadri Institute of Technology: 10 undergraduate students from the B.Tech AI&DS programme with self-reported basic music knowledge, and 10 postgraduate students from the M.Tech programme with self-reported advanced technical backgrounds. Participants completed a standardised 6-task evaluation protocol covering all major system functions: application launch and seed input, melody generation with default parameters, temperature adjustment and re-generation, piano roll interpretation, MIDI playback, and file export. Following task completion, participants independently completed a 10-item System Usability Scale (SUS) questionnaire.

Table VIII: User Acceptance Testing Results (n = 20)

Evaluation Criterion	Score / Response
System Usability Scale (SUS) Score	76.0 / 100 (Grade B, classification: Good)
Task completion rate (all 6 tasks)	94% (average across all tasks)
Temperature control — 'Understood without explanation'	85% (17 of 20 participants)
Piano roll — 'Helpful for understanding melody'	90% (18 of 20 participants)
Generated melody — 'Sounds musical and coherent'	80% (16 of 20 participants)
MIDI export — 'Successfully saved and opened in DAW'	100% (20 of 20 participants)

Overall platform recommendation rate	85% would recommend to peers
Average task completion time (6-task protocol)	8.6 minutes

5.5 Comparison with Existing Systems

Table IX presents a feature-by-feature comparison of MelodAI against the three most relevant existing melody generation systems: Magenta Basic RNN (Google Brain), MusicVAE (Google Brain), and Amper Music (a commercial AI music generation service). The comparison confirms that MelodAI uniquely delivers LSTM-based melody generation with user-accessible temperature control, piano roll visualisation, and MIDI export in a fully open-source, locally deployable platform without subscription costs.

Table IX: Comparison with Existing Music Generation Systems

Feature	MelodAI	Magenta Basic RNN	MusicVAE	Amper Music
LSTM-Based Generation	Yes	Yes	No (VAE)	Proprietary
Temperature Control	Yes	Yes	Partial	No
Seed Melody Input	Yes	Yes	Yes	No
Piano Roll Visualisation	Yes	No	No	No
MIDI Export	Yes	Yes	Yes	Yes
Open Source & Local Deploy	Yes	Yes	Yes	No (SaaS)
Free to Use	Yes	Yes	Yes	No ($)
Polyphonic Output	No	No	Yes	Yes

CHAPTER VI

TESTING & VALIDATION

6.1 Test Strategy and Types

The MelodAI system was validated through a structured testing strategy comprising four complementary test types, each targeting a different aspect of system correctness and quality.

Unit Testing validates the correctness of individual functions within each pipeline module in isolation. The note extraction function is tested against a set of known MIDI files with verified note contents to confirm accurate pitch and duration extraction. The integer encoding function is tested with a predefined vocabulary to verify correct index assignments and round-trip fidelity. The LSTM model's forward pass is tested with randomly generated input tensors to verify output shape and the validity of the softmax probability distribution (summing to 1.0 within floating-point tolerance).

Integration Testing verifies the correct data handoff between adjacent modules. The preprocessing-to-training integration test confirms that the vocabulary mapping produced during preprocessing is consistent with the input shape expected by the LSTM model. The training-to-inference integration test confirms that the model loaded from the checkpoint file produces the same output as the model held in memory after training. The inference-to-output integration test confirms that the generated note token sequence can be correctly decoded and rendered as a valid, playable MIDI file by Music21.

System Testing simulates the complete end-to-end workflow from raw MIDI corpus to playable MIDI output. A complete system test run confirms that the pipeline executes without errors across all stages, that the generated MIDI file is valid and playable in standard MIDI software, and that the graphical interface correctly reflects generation progress and provides working playback controls.

User Acceptance Testing involves subjective evaluation by the 20-participant panel described in Chapter V, assessing outputs on dimensions of musical quality, interface usability, and overall task completion success.

6.2 Sample Test Cases

Table X: Sample Test Cases

TC ID	Test Scenario	Expected Output	Actual Result	Status
TC1	Valid seed notes entered (C4 E4 G4 A4)	100-note melody MIDI file generated successfully	MIDI file generated in 2.8s	PASS
TC2	Empty seed input submitted	System uses default seed; generation proceeds normally	Default seed applied; melody generated	PASS
TC3	Temperature set to 0.5 (low creativity)	Generated melody is conservative and predictable	Low pitch variety; conventional contour observed	PASS
TC4	Temperature set to 1.5 (high creativity)	Greater note variety in generated output	Higher entropy pitch distribution confirmed	PASS
TC5	Export MIDI button clicked after generation	MIDI file saved to user-specified path	File saved; opened correctly in MuseScore	PASS
TC6	Invalid note name entered (e.g., X9)	Error message displayed; generation blocked	Error dialogue shown; no crash	PASS
TC7	Model file missing at application launch	FileNotFoundError with informative message	Exception caught; user-friendly alert shown	PASS
TC8	Generation length set to 500 notes	500-note melody generated within 15 seconds on CPU	Completed in 13.4s on Intel i7 CPU	PASS
TC9	Playback initiated during active generation	Playback button disabled until generation completes	Button correctly disabled; no race condition	PASS
TC10	Piano roll displayed after generation	Correct note sequence rendered as coloured bars	All 100 notes rendered with correct pitch mapping	PASS

6.3 Bug Fixes and Iterations

6.3.1 Vocabulary Mismatch Between Training and Inference

An early issue was identified where the vocabulary mapping serialised during training did not match the mapping loaded during inference if the training data directory contents changed between runs. This occurred because the original implementation iterated over the filesystem

directory in OS-defined order, which is non-deterministic across platforms. The bug was resolved by sorting the unique note set alphabetically before assigning integer indices, ensuring a deterministic and reproducible mapping independent of file system ordering.

6.3.2 MIDI Duration Quantisation Errors

Generated note sequences occasionally contained note durations that were non-standard musical values (e.g., 0.33333 quarter notes arising from floating-point arithmetic in the duration encoding), resulting in MIDI files with irregular timing that some software players interpreted incorrectly. This was resolved by implementing a duration quantisation step in the MIDI rendering pipeline that snaps all durations to the nearest standard musical value (0.25, 0.5, 0.75, 1.0, 1.5, 2.0, and 4.0 quarter notes) using a lookup table of allowable durations.

6.3.3 UI Thread Blocking During Generation

In the initial implementation, the melody generation loop ran on the main UI thread, causing the application window to become unresponsive and display a “not responding” status on Windows during generation. This was resolved by refactoring the generation function to execute in a background thread using Python’s threading module, with a callback that updates the UI components on the main thread upon completion. The Generate button is disabled and a progress indicator is shown for the duration of the background thread execution.

6.4 Challenges and Proposed Solutions

6.4.1 Capturing Long-Range Musical Dependencies

Challenge: The fundamental challenge in melody generation is modeling long-range dependencies — the repetition of themes, harmonic resolution, and phrase-level symmetry that give music its sense of structural coherence. Standard RNNs fail to capture these dependencies due to the vanishing gradient problem.

Solution: The MelodAI system addresses this challenge through the deliberate choice of LSTM networks, whose gated memory mechanism allows selective retention of relevant contextual information across long sequence spans. The sequence length of 100 time steps ensures that the model has sufficient context to identify and reproduce medium-range melodic patterns. Stacking two LSTM layers provides the hierarchical representational

capacity necessary to capture both fine-grained note transitions and higher-level phrase structures.

6.4.2 Avoiding Musical Monotony in Generated Outputs

Challenge: A common failure mode of generative models trained on musical data is the production of repetitive, monotonous outputs. This arises when the model converges to predicting a small set of high-probability notes regardless of context, resulting in melodies dominated by a single pitch.

Solution: The temperature-controlled sampling mechanism mitigates this problem by introducing controlled stochasticity into the note selection process. Training on a diverse corpus of folk melodies from multiple cultural traditions exposes the model to a wide variety of melodic patterns, and dropout regularisation during training prevents the model from memorising specific training examples rather than learning generalizable patterns.

6.4.3 Overfitting on a Moderate-Sized Training Corpus

Challenge: Deep learning models trained on limited musical corpora are susceptible to overfitting, wherein the model memorises specific melodies from the training data rather than learning generalizable musical patterns. An overfit model generates melodies that closely replicate training examples rather than producing novel compositions.

Solution: Dropout regularisation is applied after each LSTM layer, randomly deactivating 30% of hidden units during each training step. This forces the network to develop redundant, distributed representations of musical patterns. Model checkpointing based on validation loss ensures that the deployed model represents the best generalisation performance rather than the final epoch state which may be overfit.

6.4.4 Computational Resource Requirements for Training

Challenge: Training deep LSTM networks on large musical corpora is computationally intensive, requiring GPU-accelerated hardware and extended training periods. This poses a practical challenge for students and researchers working with limited computational resources.

Solution: The MelodAI model is designed to be sufficiently compact to permit training on a single consumer-grade GPU within approximately three hours for 50 epochs. Pre-trained model weights are distributable alongside the repository, enabling deployment without retraining. TensorFlow's optimised CUDA kernels ensure efficient GPU utilisation, and the batch training architecture with ModelCheckpoint callbacks allows training to be interrupted and resumed without loss of progress.

CHAPTER VII

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

MelodAI successfully demonstrates that a student-developed, open-source deep learning system can deliver musically meaningful and technically rigorous AI-driven melody generation. The platform integrates a Music21-based MIDI preprocessing pipeline, a stacked LSTM model trained on the ESAC folk melody corpus, a temperature-controlled autoregressive inference engine, a MIDI output rendering pipeline, and a user-friendly graphical interface into a single cohesive system that generates original melodic compositions in near real-time on standard desktop hardware.

The experimental evaluation validates each core component with appropriate rigour. The LSTM model achieves a training loss of 0.85 and a validation loss of 1.10 after 50 training epochs, with a training accuracy of 78.4% and a validation accuracy of 64.2% on the next-note prediction task — performance that substantially exceeds the random baseline and confirms that the model has learned meaningful statistical patterns governing musical note sequences. Generated melodies exhibit a pitch distribution entropy of 3.82 bits, a repeated motif rate of 23.4%, and a low consecutive repeat rate of 4.1%, collectively demonstrating that outputs are melodically varied, structurally coherent, and free from the monotonous repetition that characterises lower-quality generative models.

System performance benchmarks confirm sub-3-second melody generation latency on mid-range CPU hardware, satisfying the interactive real-time target established in the system objectives. User Acceptance Testing yields a System Usability Scale score of 76.0 (Grade B, classification: Good), confirming that the complexity of the underlying deep learning model has been successfully abstracted into an accessible, intuitive interface that all 20 test participants could operate without prior training. The 100% MIDI export success rate confirms the practical utility of the system for downstream music production workflows.

The broader contribution of this work is the demonstration that accessible, high-quality AI music generation tools can be built using freely available open-source frameworks — TensorFlow, Keras, and Music21 — without institutional resources or proprietary platforms,

making the system reproducible and extendable by any student or researcher with a standard laptop and an internet connection.

7.2 Future Scope

- Transformer-Based Architecture: Integration of a Music Transformer with relative positional attention mechanisms, as proposed by Huang et al. (2018), is the most impactful near-term enhancement. Initial offline experiments suggest that a GRU-Attention hybrid model would substantially improve the structural coherence of generated long-form compositions by enabling explicit modeling of dependencies between arbitrary positions in the input sequence.

- Polyphonic Music Generation: Extending MelodAI from monophonic single-voice generation to polyphonic multi-voice generation — supporting chord accompaniment, bass lines, and multi-instrument arrangements — would dramatically expand the system's creative capabilities and its practical utility for music production applications.

- Conditional Style Generation: Incorporating conditional generation mechanisms that allow users to specify a target musical style, genre, key, or emotional quality (e.g., happy, melancholic, energetic) through learned style embeddings or conditioning tokens would provide finer control over the character of generated compositions.

- Real-Time Interactive Composition: Developing a live performance mode in which MelodAI generates and plays melodies in real-time response to MIDI input from a keyboard performer would enable collaborative human-AI improvisation, with applications in live music performance and interactive art installations.

- Music Therapy Applications: Integrating MelodAI with biometric sensors and emotional state classifiers to generate adaptive musical compositions tailored to the physiological and psychological state of individual listeners holds significant promise for music therapy applications, particularly in the treatment of anxiety, depression, and neurological conditions.

- Web-Based Deployment: Developing a cloud-hosted web application that allows users to access MelodAI from any device through a standard web browser, without requiring local Python installation, would substantially expand the system's accessibility and reach to the broader community of music enthusiasts and researchers.

- Large-Scale Pretraining: Training MelodAI on a significantly expanded corpus spanning classical, jazz, blues, and world music traditions through large-scale pretraining would enrich the model's musical vocabulary and enable stylistically diverse generation that transcends the folk melody style of the current ESAC training corpus.

REFERENCES

[1] D. Eck and J. Schmidhuber, "A First Look at Music Composition Using LSTM Recurrent Neural Networks," Technical Report IDSIA-07-02, IDSIA, 2002.

[2] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[3] N. Boulanger-Lewandowski, Y. Bengio, and P. Vincent, "Modeling Temporal Dependencies in High-Dimensional Sequences: Application to Polyphonic Music Generation and Transcription," in Proceedings of the 29th International Conference on Machine Learning (ICML), 2012.

[4] E. Waite, "Generating Long-Term Structure in Songs and Stories," Magenta Blog, Google Brain, 2016. [Online]. Available: https://magenta.tensorflow.org.

[5] A. Roberts, J. Engel, C. Raffel, C. Hawthorne, and D. Eck, "A Hierarchical Latent Vector Model for Learning Long-Term Structure in Music," in Proceedings of the 35th International Conference on Machine Learning (ICML), 2018.

[6] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, "Music Transformer: Generating Music with Long-Term Structure," in Proceedings of the 7th International Conference on Learning Representations (ICLR), 2019.

[7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), pp. 5998–6008, 2017.

[8] L.-C. Yang, S.-Y. Chou, and Y.-H. Yang, "MidiNet: A Convolutional Generative Adversarial Network for Symbolic-Domain Music Generation," in Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR), 2017.

[9] H.-W. Dong, W.-Y. Hsiao, L.-C. Yang, and Y.-H. Yang, "MuseGAN: Multi-Track Sequential Generative Adversarial Networks for Symbolic Music Generation and Accompaniment," in Proceedings of the 32nd AAAI Conference on Artificial Intelligence, 2018.

[10] M. Cuthbert and C. Ariza, "Music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data," in Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR), pp. 637–642, 2010.

[11] V. Velardo, "Generating Melodies with RNN-LSTM: Series Overview," Valerio Velardo — The Sound of AI, 2021. [Online]. Available: https://github.com/musikalkemist.

[12] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," in Advances in Neural Information Processing Systems (NeurIPS), pp. 2672–2680, 2014.

[13] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A Generative Model for Raw Audio," arXiv preprint arXiv:1609.03499, 2016.

[14] F. Chollet, "Keras: The Python Deep Learning API," 2015. [Online]. Available: https://keras.io.

[15] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), pp. 265–283, 2016.

[16] J. A. Nielsen, "Usability Engineering," Academic Press, 1993.

[17] H. H. Mao, T. Shin, and G. Cottrell, "DeepJ: Style-Specific Music Generation," in Proceedings of the 9th IEEE International Conference on Humanoid Robots (Humanoids), 2018.

[18] O. Mogren, "C-RNN-GAN: Continuous recurrent neural network with adversarial training," in Proceedings of the Constructive Machine Learning Workshop at NeurIPS, 2016.

[19] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, "Neural Audio Synthesis of Musical Notes with WaveNet Autoencoders," in Proceedings of the 34th International Conference on Machine Learning (ICML), 2017.

ISSN: 2582-3930

ACCEPTANCE CERTIFICATE

Impact Factor: 8.659
DOI Prefix: 10.55041

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT (IJSREM)

Peer-Reviewed Journal || An Open Access Scholarly Journal || Indexed in Major Databases

We are pleased to inform you that your manuscript titled

MelodAI – Composing the Future of Music with Artificial Intelligence

has been ACCEPTED for publication in

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT (IJSREM)

VOLUME 10 ISSUE 03 MARCH- 2026

We are delighted to see your commitment & hardwork to share your research is being recognized. We look foreward to helping you with all of your publication needs. Thank you for choosing IJSREM!!

Editor-in-chief
IJSREM

www.ijsrem.com

e-mail: editor@ijsrem.com

DOI: 10.55041/IJSREM58837

ISSN: 2582-3930
Impact Factor: 8.659

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering & Management is hereby awarding this certificate to

K. Sandhya Rani

In recognition to the publication of the paper titled

MelodAI – Composing the Future of Music with Artificial Intelligence

published in IJSREM Journal on Volume 10 Issue 04 April, 2026

Editor-in-Chief
IJSREM Journal

www.ijsrem.com

e-mail: editor@ijsrem.com

DOI: 10.55041/IJSREM58837

ISSN: 2582-3930
Impact Factor: 8.659

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering & Management is hereby awarding this certificate to

Gangireddy Charishma

In recognition to the publication of the paper titled

MelodAI – Composing the Future of Music with Artificial Intelligence

published in IJSREM Journal on Volume 10 Issue 04 April, 2026

Editor-in-Chief
IJSREM Journal

www.ijsrem.com

e-mail: editor@ijsrem.com

DOI: 10.55041/IJSREM58837

ISSN: 2582-3930
Impact Factor: 8.659

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering & Management is hereby awarding this certificate to

Cheerladinne Sai Deepthi

In recognition to the publication of the paper titled

MelodAI – Composing the Future of Music with Artificial Intelligence

published in IJSREM Journal on Volume 10 Issue 04 April, 2026

Editor-in-Chief
IJSREM Journal

www.ijsrem.com

e-mail: editor@ijsrem.com

DOI: 10.55041/IJSREM58837

ISSN: 2582-3930
Impact Factor: 8.659

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering & Management is hereby awarding this certificate to

Bandaru Naga Rohnitha

In recognition to the publication of the paper titled

MelodAI – Composing the Future of Music with Artificial Intelligence

published in IJSREM Journal on Volume 10 Issue 04 April, 2026

Editor-in-Chief
IJSREM Journal

www.ijsrem.com

e-mail: editor@ijsrem.com

DOI: 10.55041/IJSREM58837

ISSN: 2582-3930
Impact Factor: 8.659

INTERNATIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING & MANAGEMENT

An Open Access || Peer-Reviewed Scholarly Journal || Indexed in Major Databases

CERTIFICATE OF PUBLICATION

International Journal of Scientific Research in Engineering & Management is hereby awarding this certificate to

Gugulothu Koteswara Rao Naik

In recognition to the publication of the paper titled

MelodAI – Composing the Future of Music with Artificial Intelligence

published in IJSREM Journal on Volume 10 Issue 04 April, 2026

Editor-in-Chief
IJSREM Journal

www.ijsrem.com

e-mail: editor@ijsrem.com

International Journal of Scientific Research in Engineering and Management (IJSREM)
Volume: 10 Issue: 04 | April - 2026 SJIF Rating: 8.659 ISSN: 2582-3930

MelodAI – Composing the Future of Music with Artificial Intelligence

K. SANDHYA RANI
Assistant professor

Gangireddy Charishma
22BQ1A5424

Cheerladinne Sai Deepthi
22BQ1A5424

Bandaru Naga Rohnitha
22BQ1A5413

Gugulothu Koteswara Rao Naik
22BQ1A54551

Department of Artificial Intelligence and Data Science
Vasireddy Venkatadri Institute of Technology (Autonomus)
Affiliated to JNTUK | NAAC 'A' Grade | Guntur, Andhra Pradesh — 522508

Abstract—Music composition has traditionally been an endeavour requiring significant human expertise, creativity, and an in-depth understanding of musical theory. The rapid advancement of deep learning methodologies has opened transformative possibilities for automating creative tasks, including music generation. This paper presents MelodAI, an artificial intelligence-driven system designed to generate original musical compositions by learning temporal patterns from existing MIDI-based musical datasets. The proposed system employs Long Short-Term Memory (LSTM) networks, a specialized class of Recurrent Neural Networks (RNN) capable of modelling long-range sequential dependencies, to capture and reproduce harmonic and melodic structures. MelodAI processes encoded MIDI data, trains a multi-layered LSTM model to learn note sequences and chord progressions, and generates novel compositions that are musically coherent and contextually relevant. The system achieves a training accuracy of approximately 94.7% and demonstrates strong qualitative performance through human evaluation studies. Experimental results indicate that the proposed architecture consistently outperforms baseline Hidden Markov Model and basic RNN approaches in generating structured and melodically appealing music. The system is capable of real-time generation and outputs standard MIDI files, making it immediately useful for entertainment, therapy, gaming, and education.

Index Terms—*Music Generation, Long Short-Term Memory (LSTM), Deep Learning, MIDI Processing, Recurrent Neural Networks, Sequence Prediction, Music21, Artificial Intelligence.*

I. INTRODUCTION

The convergence of artificial intelligence and human creativity represents one of the most compelling frontiers of contemporary research. Among the various domains of creative intelligence, music generation stands out as particularly challenging due to the multi-dimensional nature of musical structure, encompassing melody, harmony, rhythm, timbre, and temporal dynamics. The question of whether a machine can compose music that is not only syntactically correct but also aesthetically meaningful has attracted significant scholarly and industrial interest over the past two decades.

Traditional music composition demands years of formal training, a refined understanding of music theory, and a deep intuitive grasp of emotional resonance. These requirements create a high barrier to entry for aspiring composers, particularly in domains such as background scoring for games, films, and therapeutic environments, where demand for custom music far exceeds the availability of professional composers. Automated music generation systems offer a promising solution by enabling on-demand, customizable, and contextually appropriate composition.

Early approaches to algorithmic music composition relied on rule-based systems and probabilistic models such as Markov chains and Hidden Markov Models (HMM). While these methods produced structurally valid sequences, they frequently failed to capture the long-range dependencies and nuanced patterns that characterize meaningful musical phrases. The advent of deep learning, particularly Recurrent Neural Networks (RNN) and their more advanced variant, Long Short-Term Memory (LSTM) networks, has dramatically altered this landscape.

This paper introduces MelodAI, a deep learning-based music generation system that leverages LSTM networks trained on MIDI (Musical Instrument Digital Interface) encoded datasets. MelodAI is designed to learn the structural and stylistic characteristics of musical compositions and generate novel sequences that are harmonically coherent and melodically engaging. The system incorporates a user-friendly interface and outputs standard MIDI files for immediate playback and editing.

The remainder of this paper is organized as follows: Section II presents a review of existing literature; Section III defines the problem statement; Section IV outlines the objectives; Section V describes the methodology; Section VI explains the system architecture; Section VII discusses implementation details; Section VIII presents results; Sections IX–XII address applications, advantages, limitations, and future scope; and Section XIII concludes the paper.

II. LITERATURE REVIEW

The field of algorithmic and AI-assisted music composition has a rich history spanning several decades. Early symbolic approaches, such as those pioneered by Hiller and Isaacson in the Illiac Suite (1957), used rule-based methods to generate music conforming to classical counterpoint rules [1]. Subsequent decades saw the application of probabilistic models, most notably Markov chains, which modelled state transitions between musical notes. While computationally tractable, these models suffered from fixed context windows and an inability to capture global musical structure [2].

The introduction of neural networks into music generation offered a significant paradigm shift. Mozer [3] was among the first to apply recurrent neural networks to music composition, demonstrating that RNNs could learn temporal dependencies in musical sequences. However, vanilla RNNs suffered from the well-documented vanishing gradient problem, limiting their effective memory horizon. The

seminal work of Hochreiter and Schmidhuber [4] introduced LSTM networks in 1997, which addressed this limitation through input, output, and forget gates that regulate information flow through the network.

The application of LSTMs to music generation was explored extensively by Johnson [5] in the Magenta project by Google Brain, demonstrating that deep LSTM models could generate piano music with notable structural coherence. Briot et al. [6] provided a comprehensive survey of deep learning approaches in music generation, identifying LSTM-based architectures among the most consistently performant. More recently, attention-based architectures, particularly the Transformer model [7], have been applied with considerable success. Huang et al. [8] proposed Music Transformer, employing relative attention for extended piano compositions. OpenAI's MuseNet [9] demonstrated that large-scale Transformer models trained on diverse musical corpora can generate multi-instrument compositions.

A comparative overview of key existing systems is presented in Table I. As indicated, prior deep learning systems are either computationally expensive, domain-restricted, or limited in output controllability. MelodAI addresses these gaps by combining LSTM sequence modelling with a lightweight, accessible design optimized for real-world deployment.

TABLE I: Comparison of Existing Music Generation Systems

System	Model	Output	Coherence	Access
Mozer 1994 [3]	Vanilla RNN	Mono	Low	High
Magenta [5]	Deep LSTM	Piano	Moderate	Moderate
Music Transformer [8]	Transformer	Piano	High	Low
MuseNet [9]	GPT-2	Multi	Very High	Very Low
MelodAI (Ours)	3-Layer LSTM	MIDI	High	High

III. PROBLEM STATEMENT

Despite the growing demand for original music content across entertainment, education, and therapeutic domains, several critical challenges impede accessibility and scalability of music composition:

- Traditional music composition demands years of expert training, creating significant barriers for non-professional users and developers of interactive systems.
- Existing AI models for music generation, particularly older probabilistic models, frequently lack melodic coherence, producing sequences that are locally plausible but globally inconsistent.
- Manual identification of musical patterns in large corpora is both time-intensive and subjective, making scalable data-driven analysis difficult.
- Industries such as gaming, film production, and digital therapeutics require rapid, customizable music generation without the overhead of commissioning professional composers.
- Many existing deep learning systems require substantial computational infrastructure, limiting deployment for smaller institutions and independent developers.

IV. OBJECTIVES

The primary and secondary objectives of this research are as follows:

- To design and implement an end-to-end AI-driven system for generating original music compositions using deep learning techniques.
- To leverage Long Short-Term Memory (LSTM) networks for sequence modelling of musical notes, chords, and durations extracted from MIDI files.
- To preprocess and encode raw MIDI musical data into structured numerical representations suitable for neural network training.
- To train and optimize a multi-layered LSTM model through hyperparameter tuning to achieve high generation accuracy.
- To generate musically coherent MIDI output playable in real time and editable using standard music production tools.
- To evaluate the system against baseline models using quantitative metrics and qualitative human evaluation studies.

V. METHODOLOGY

A. Data Collection

The MelodAI system is trained on a curated corpus of MIDI files sourced from the Lakh MIDI Dataset [10], comprising over 176,000 unique MIDI files, and the Classical Music MIDI Archive. A subset of approximately 2,500 MIDI files across three genres—classical piano, jazz, and ambient electronic—was selected. This multi-genre dataset ensures the model learns generalized musical patterns rather than overfitting to genre-specific conventions. All selected files were verified for structural completeness, and corrupted or incomplete MIDI files were discarded during initial data audit.

B. Data Preprocessing

MIDI files encode music as event sequences rather than raw audio, providing a structured symbolic representation well-suited for sequential modelling. The Music21 Python library [11] was employed to parse MIDI files and extract notes, chords, and rest durations. Each note was encoded by pitch value and duration; chords were represented as sorted tuples of constituent pitches. The full vocabulary of unique musical elements was compiled and assigned integer indices.

Input sequences of fixed length N= 100 were constructed by sliding a window across each tokenized note stream. The corresponding target was the immediately

subsequent element. All inputs were normalized to [0, 1] by dividing indices by vocabulary size. One-hot encoding was applied to target labels for categorical cross-entropy computation. The dataset was partitioned into training (80%), validation (10%), and test (10%) subsets using stratified sampling.

C. Model Architecture – LSTM Networks

The Long Short-Term Memory network, introduced by Hochreiter and Schmidhuber in 1997 [4], is a specialized variant of the Recurrent Neural Network designed to overcome the vanishing gradient problem. The fundamental innovation lies in its memory cell and three gating mechanisms: the forget gate, the input gate, and the output gate. Together, these mechanisms allow the LSTM cell to selectively retain information over arbitrarily long sequences—a property critical in music, where motifs established early in a composition may recur hundreds of time steps later.

The MelodAI architecture consists of a stacked LSTM network comprising: an Embedding layer mapping integer-encoded note indices to dense 256-dimensional vectors, followed by three LSTM layers with 512, 512, and 256 units respectively. Dropout layers with a rate of 0.3 are interleaved between LSTM layers to prevent overfitting. A fully connected Dense layer with softmax activation produces a probability distribution over the musical vocabulary. The full architecture is summarized in Table II.

TABLE II: MelodAI LSTM Model Architecture

L#	Layer / Config	Output	Params
1	Embedding (Vocab×256)	(100, 256)	Learnable
2	LSTM (512, ret_seq=T)	(100, 512)	1,574,912
3	Dropout (0.3)	(100, 512)	0
4	LSTM (512, ret_seq=T)	(100, 512)	2,099,200
5	Dropout (0.3)	(100, 512)	0
6	LSTM (256 units)	(256)	787,456
7	Dense (Vocab, Softmax)	(Vocab)	Variable

D. Training Process

The model was trained using the Adam optimizer [12] with an initial learning rate of 0.001 and a batch size of 64 over 200 epochs. Categorical cross-entropy was employed as the loss function. A ReduceLROnPlateau callback reduced the learning rate by a factor of 0.5 when validation loss stagnated over 10 consecutive epochs. Model checkpointing retained weights corresponding to minimum validation loss. Training was performed on an NVIDIA RTX 3080 GPU using TensorFlow 2.9. Gradient clipping with a threshold of 1.0 was applied to stabilize early training phases.

E. Music Generation Process

During inference, the trained model receives a seed sequence of N=100 elements randomly sampled from the test corpus. A temperature-controlled sampling strategy is employed: a temperature parameter τ scales the logits before softmax. Lower τ (e.g., 0.5) produces conservative outputs while higher values (e.g., 1.2) introduce creative variability. For primary experiments, $\tau = 0.9$ was selected as a balance between coherence and novelty. Generated tokens are decoded into a Music21 stream and exported as a standard MIDI file.

VI. SYSTEM ARCHITECTURE

The MelodAI system is composed of five principal functional modules operating in a sequential pipeline. The Data Ingestion Module accepts MIDI files and interfaces with Music21 to extract musical primitives. The Preprocessing Module converts these primitives into integer-encoded sequences, constructs the vocabulary, generates training windows, and applies normalization and one-hot encoding. The Model Training Module manages LSTM construction, compilation, and optimization, incorporating callbacks for learning rate scheduling and checkpointing.

The Generation Module performs iterative next-note prediction from a user-defined or randomly selected seed and assembles the output token stream. The Output Module decodes tokens into a Music21 score, applies post-processing such as quantization and tempo assignment, and exports the MIDI file. Users interact through a Flask-based web interface exposing genre selection, sequence length, and temperature controls. This frontend-backend separation ensures the deep learning pipeline can be independently scaled without modifying the user-facing application.

VII. IMPLEMENTATION DETAILS

The MelodAI system was implemented entirely in Python 3.9 using the following technology stack:

- TensorFlow 2.9 / Keras: Primary deep learning framework for constructing, training, and serializing the LSTM model via the Keras Sequential API.
- Music21 (v7.3): Toolkit for MIDI parsing, musical element extraction, and score reconstruction, providing support for both monophonic and polyphonic structures.
- NumPy and Pandas: Efficient numerical array manipulation, sequence windowing, and dataset management during preprocessing.
- Scikit-learn: Dataset splitting with stratified sampling to maintain genre distribution across training, validation, and test subsets.
- Flask (v2.1): Lightweight web framework managing HTTP routing and API endpoints for generation requests.
- Pygame (v2.1): In-browser MIDI playback functionality enabling users to audition generated compositions within the interface.

The trained model weights are serialized in HDF5 format and reloaded by the generation module upon user request. Generated MIDI files are stored in an output directory and served via a download endpoint.

VIII. RESULTS AND DISCUSSION

The MelodAI system was evaluated using quantitative performance metrics and a structured human evaluation study. Quantitative metrics included training accuracy, validation accuracy, categorical cross-entropy loss, and perplexity on the held-out test set. Qualitative evaluation was conducted through a listener study involving 30 participants with varying levels of musical training, rating compositions on melodic coherence, rhythmic regularity, and overall musicality on a 5-point Likert scale.

TABLE III: Quantitative Comparison of Music Generation Models

Model	Tr. Acc.	Val. Acc.	Perplexity	Human (/5)
HMM (Baseline)	72.4%	68.1%	38.6	2.4
Vanilla RNN	81.3%	76.5%	22.3	2.9
1-Layer LSTM	88.7%	85.2%	14.8	3.5
MelodAI (Ours)	94.7%	92.1%	8.2	4.2

The results in Table III demonstrate a clear performance hierarchy. The HMM baseline achieved the lowest accuracy and highest perplexity, reflecting its fundamental limitation in capturing long-range dependencies. The vanilla RNN showed marginal improvement but remained hampered by vanishing gradients over sequences of length 100. The single-layer LSTM achieved a substantial improvement, validating gated memory mechanisms for musical sequence modelling. The proposed MelodAI three-layer LSTM achieved the best results, with a test accuracy of 92.1% and perplexity of 8.2. Human evaluators assigned MelodAI an average of 4.2/5, with particularly high ratings in melodic coherence (4.4) and rhythmic regularity (4.3).

Training and validation loss decreased smoothly over 200 epochs without significant divergence, confirming that the applied regularization strategies successfully mitigated overfitting. The model demonstrated sensitivity to the temperature parameter: at $\tau = 0.9$, the most musically satisfying results were obtained, consistent with subjective evaluations. Qualitatively, generated compositions exhibited tonal centre establishment, motif repetition, and phrase-level structure aligned with training genre conventions.

TABLE IV: Hyperparameter Tuning Results for MelodAI

Trial	Emb. Dim	LSTM Units	Dropout	LR	Accuracy
1	256	512/512/256	0.3	0.001	0.9470
2	256	256/256/128	0.4	0.001	0.9312
3	128	512/256	0.3	0.0005	0.9218
4	128	256/256	0.5	0.001	0.9081

IX. APPLICATIONS

The MelodAI system has broad applicability across diverse industries:

- Gaming and Interactive Media: Real-time adaptive background music generation responding to in-game states and player actions without reliance on manually curated soundtracks.
- Music Therapy: Generation of personalized, mood-appropriate musical sequences for clinical therapeutic applications, including management of anxiety, depression, and neurodevelopmental conditions.
- Film and Advertising Post-Production: Rapid prototyping of background scores and jingles during early production phases, reducing dependency on commissioned composers.
- Music Education: A creative tool for students to explore harmonic possibilities, understand compositional structure, and experiment with AI-assisted arrangement.
- Personal Content Creation: Enabling hobbyists, podcasters, and content creators to produce original royalty-free music for personal and commercial projects.
- Assistive Technology: Supporting individuals with impairments that limit traditional instrument playing by providing an accessible avenue for musical self-expression.

X. ADVANTAGES

- Structured Sequence Learning: The multi-layer LSTM captures both short-range intervallic relationships and long-range harmonic dependencies, producing compositions with measurable structural coherence.
- Generalizability Across Genres: Training on a multi-genre corpus enables stylistically varied outputs based on the choice of seed sequence.
- Real-Time Generation: The inference pipeline produces MIDI outputs in under two seconds on consumer-grade hardware, facilitating interactive applications.
- MIDI Compatibility: Standard MIDI output ensures immediate compatibility with professional DAWs including Ableton Live, FL Studio, and GarageBand.
- Accessibility: No musical training is required from the end user, democratizing creative music production for a wide audience.

XI. LIMITATIONS

- Monophonic Bias: The primary model is optimized for single-voice melodic generation; polyphonic generation requires more complex multi-stream architectures.
- Dataset Dependency: Output quality is bounded by the training corpus; biases present in data may manifest as stylistic tendencies in generated music.
- Lack of High-Level Structural Planning: The LSTM operates at individual note prediction level and does

not incorporate mechanisms for planning high-level forms such as verse-chorus structures or key modulations.

- Emotional Specificity: The system does not support explicit conditioning on emotional or mood parameters, limiting applicability in emotionally targeted contexts.

XII. FUTURE SCOPE

- Transformer-Based Architecture: Integration of Music Transformer [8] with relative positional attention is expected to yield further improvements in long-range structural coherence.
- Conditional Generation: Incorporating conditioning vectors for genre, mood, tempo, and instrumentation will enable fine-grained stylistic control.
- Multi-Instrument Polyphony: Extending the architecture to model multiple simultaneous voice streams will enable full orchestral arrangement generation.
- Reinforcement Learning from Human Feedback (RLHF): Incorporating human aesthetic judgements as reward signals could substantially improve subjective quality of generated compositions.
- Real-Time Audio Synthesis: Integrating with neural audio synthesis models such as WaveNet [13] will enable high-fidelity audio output directly without external MIDI synthesizers.

XIII. CONCLUSION

This paper has presented MelodAI, a deep learning-based music generation system employing a multi-layered Long Short-Term Memory architecture to learn and reproduce musical patterns from MIDI-encoded training corpora. The system was evaluated against HMM and RNN baselines and demonstrated superior performance, achieving a test accuracy of 92.1%, a perplexity of 8.2, and an average human evaluation score of 4.2 out of 5. These results validate the effectiveness of deep LSTM architectures for capturing the complex temporal dependencies inherent in musical sequences.

MelodAI addresses a genuine and growing need for accessible, intelligent music generation tools applicable across entertainment, therapy, education, and content creation. By combining robust deep learning sequence modelling with a practical MIDI output pipeline and a user-friendly interface, the system demonstrates that AI-driven music composition is technically feasible and practically valuable. The modularity of the architecture ensures that future enhancements, including Transformer models and conditional generation, can be integrated without systemic redesign. Future research will focus on expanding generative range, improving structural planning, and integrating real-time audio synthesis.

ACKNOWLEDGEMENTS

The authors gratefully acknowledge the Department of Computer Science and Engineering, National Institute of Technology, Bengaluru, for providing computational infrastructure. The authors also acknowledge the creators of the Lakh MIDI Dataset and the Music21 library for their invaluable open-source contributions to music information retrieval and computer-aided musicology.

REFERENCES

[1] L. A. Hiller and L. M. Isaacson, Experimental Music: Composition with an Electronic Computer. New York: McGraw-Hill, 1959.

[2] J. Allan, “Markov models for music,” in Proc. ICMC, 1998.

[3] M. C. Mozer, “Neural network music composition by prediction,” Connection Science, vol. 6, no. 2–3, pp. 247–280, 1994.

[4] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[5] D. Johnson, “Generating polyphonic music using tied parallel networks,” in Proc. EvoMUSART, 2017, pp. 128–143.

[6] J.-P. Briot, G. Hadjeres, and F. Pachet, “Deep learning techniques for music generation – A survey,” arXiv:1709.01620, 2017.

[7] A. Vaswani et al., “Attention is all you need,” in Advances in NIPS, vol. 30, 2017.

[8] C.-Z. A. Huang et al., “Music Transformer: Generating music with long-term structure,” in Proc. ICLR, 2019.

[9] C. Payne, “MuseNet,” OpenAI Blog, Apr. 2019. [Online]. Available: https://openai.com/blog/musenet

[10] C. Raffel, “Learning-based methods for comparing sequences,” Ph.D. dissertation, Columbia University, 2016.

[11] M. S. Cuthbert and C. Ariza, “Music21: A toolkit for computer-aided musicology,” in Proc. ISMIR, 2010, pp. 637–642.

[12] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.

[13] A. v. d. Oord et al., “WaveNet: A generative model for raw audio,” arXiv:1609.03499, 2016.